

学号 20201008303 姓名 王韵菲

使用 ADS 1.2 软件仿真，单步、全速运行程序，设置断点，打

开寄存器窗口 (Processor Registers) 监视 R0、R1 的值，打开存储器观察窗口 (Memory) 监视 0x40003100 上的值。

实验步骤：

(1) 启动 ADS 1.2，使用 ARM Executable Image 工程模板建立一个工程 Instruction1。

(2) 建立汇编源文件 TEST2.S，编写实验程序，然后添加到工程。

(3) 设置工程链接地址 R0 Base 为 0x40000000，RW Base 为 0x40003000。设置调试入口地址 Image entry point 为 0x40000000。

(4) 编译链接工程，选择【Project】->【Debug】，启动 AXD 进行软件仿真调试。

(5) 打开寄存器窗口 (Processor Registers)，选择 Current 项监视 R0，R1 的值。打开存储器观察窗口 (Memory)，设置观察地址为 0x40003100，显示方式 Size 为 32Bit，监视 0x40003100 地址上的值。

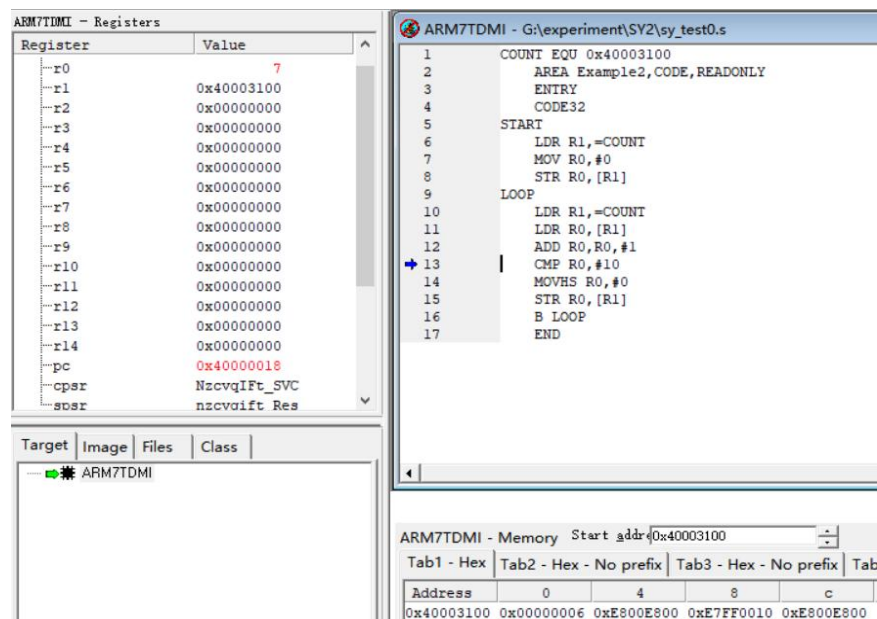


图 1 运行结果图

2、实验练习 1

实验内容：在 ADS 下用汇编语言实现 128 位数的减法。其中第一个 128 位数由高到低位于 R7~R4 中，第二个 128 位数由高到低位于 R11~R8 中。

实验步骤：

SUBS R0, R4, R8 ;将 R4 寄存器的值减去 R8 寄存器的值，并将结果存储在 R0 寄存器中。

SBCS R1, R5, R9 ;将 R5 寄存器的值减去 R9 寄存器的值，并考虑上一次减法操作的借位，结果存储在 R1 寄存器中。

SBCS R2, R6, R10 ;将 R6 寄存器的值减去 R10 寄存器的值，并考虑上一次减法操作的借位，结果存储在 R2 寄存器中。

SBC R3, R7, R11 ;将 R7 寄存器的值减去 R11 寄存器的值，并考虑上一次减法操作的借位，结果存储在 R3 寄存器中。SBC 指令不会设置条件标志位。

实验结果如下图所示：

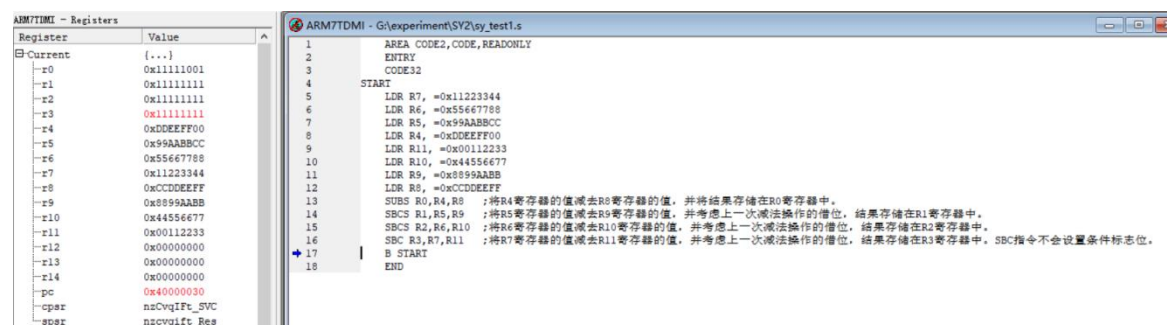


图 2 程序代码注释及运行结果图

3、实验练习 2

实验内容：在 ADS 下用汇编语言编写一个程序实现如下目的：从源地址 src 拷贝 num 个字(num*4 字节)的数据到目的地址 dst 中。

实验程序代码及注释如下图所示：

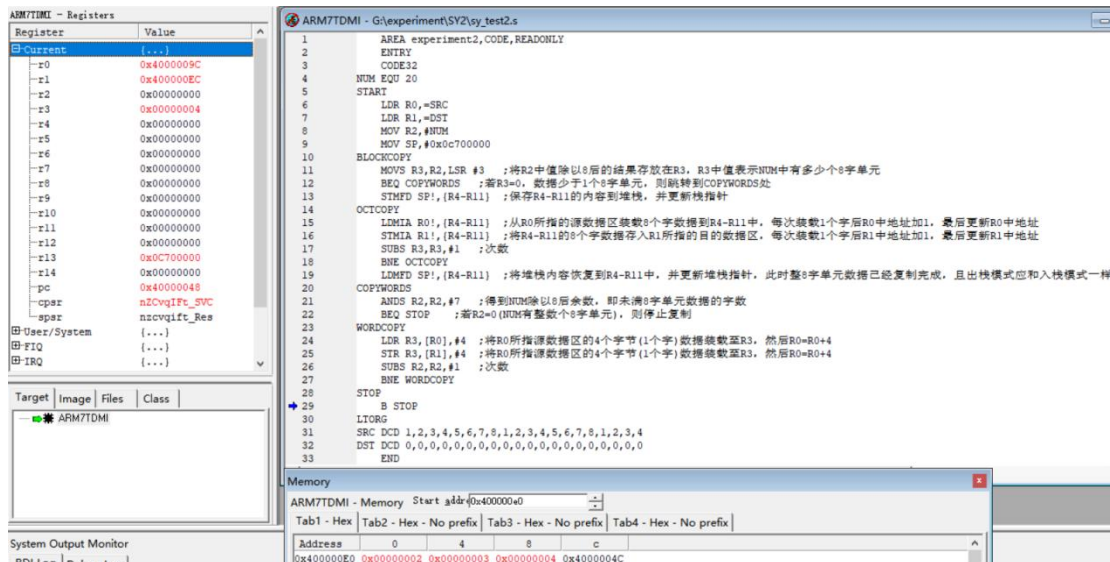


图 3 代码注释图

进行块复制，R4-R11 寄存器读取 R0 中 8 个字，并存储于 R1 中，存储器存放起始地址为 0x4000009C, 如下图所示：

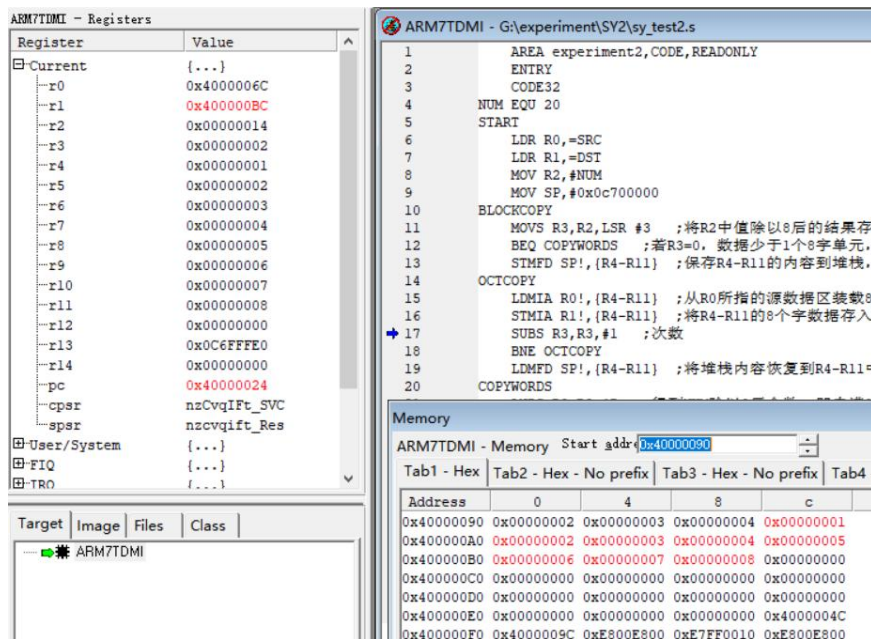


图 4 块复制

不满8字的数据一个字一个字的读取，下图为读取块元素读取后的第一个字，并将其存入起始地址 0x400000DC 中

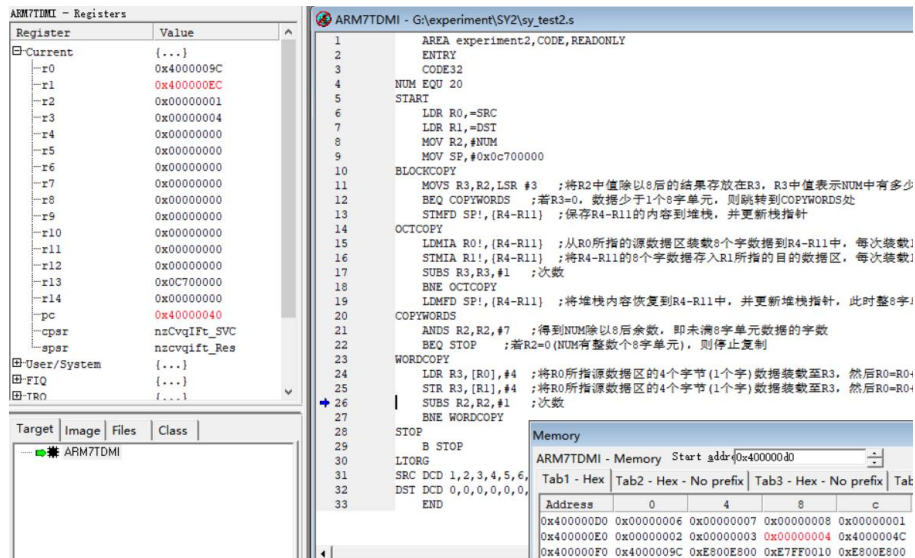


图5 逐字复制

4、实验练习 3

实验内容：根据 r0 中存放的数据 N, 实现 $1+2+3+\dots+N$

实验中寄存器 R1 存放总和，通过一个循环则可以实现，如下图

所示：

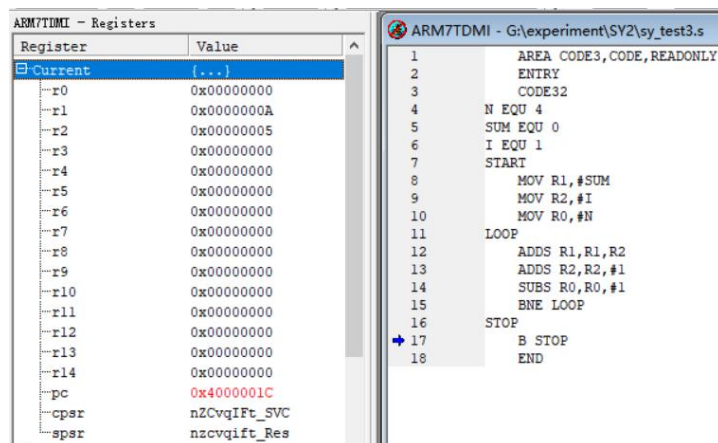


图6 求总和

5、实验练习 4

实现程序：使用 LDR 指令读取 0x30003000 地址上的数据，测试该数据的符号，若为正数，则利用 STR 指令向 0x30003100 单元写入 1；若为负数，则向该单元写入-1；若为 0，则向该单元写入 0。

大于 0 的情况如下，在 0x30003100 单元可以查看到值为 1：

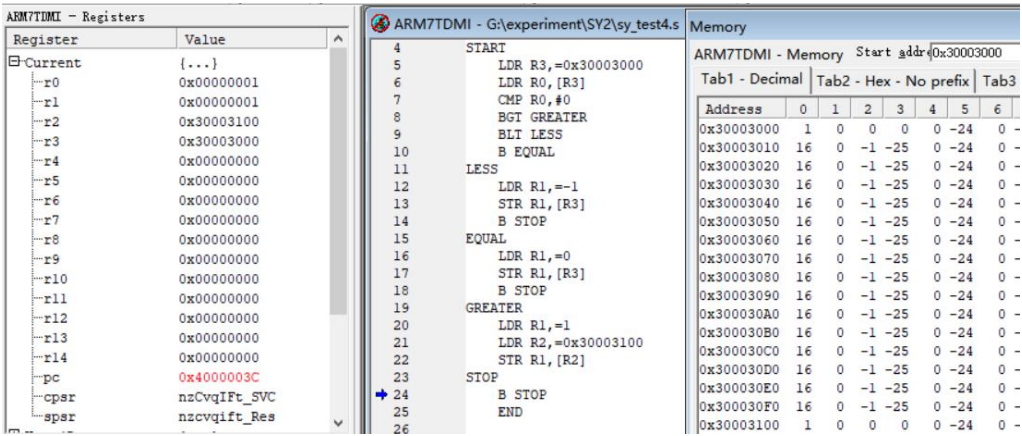


图 7 大于 0

小于 0 的情况如下，在 0x30003000 单元可以查看到值为-1：

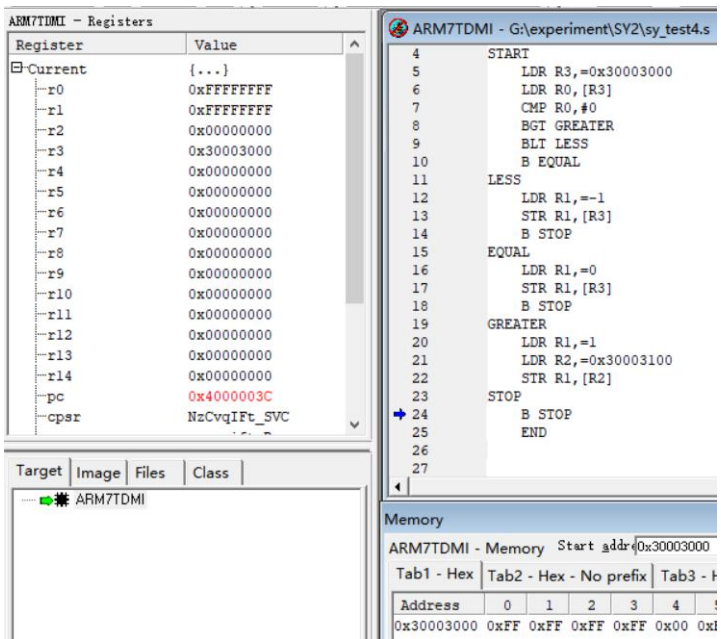


图 8 小于 0

等于 0 的情况如下，在 0x30003000 单元可以查看到值为 0：

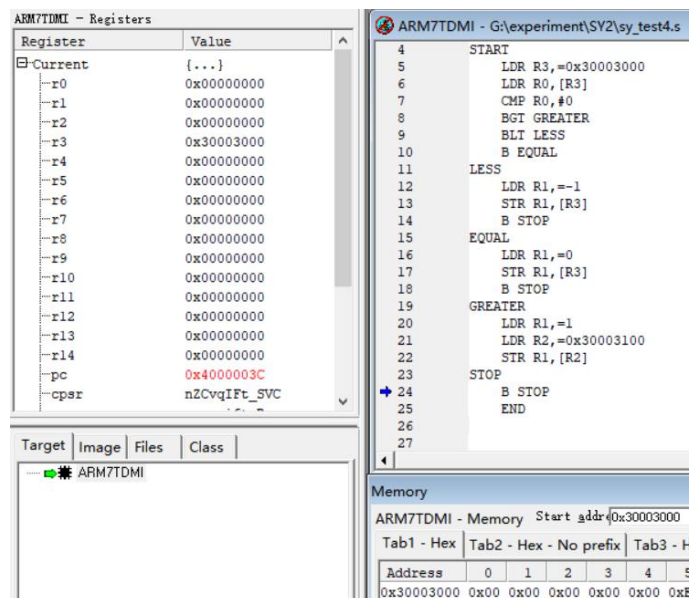


图 9 等于 0

四、 实验心得

1、 写出自己的收获

通过实验中使用的不同指令（如 LDR、STR、数据处理指令等），我更深入地理解这些指令的功能、用法和操作数的选择。同时，使用 LDR 和 STR 指令读取和写入存储器中的数据，我学会了如何正确访问内存地址，并将数据从存储器加载到寄存器中进行处理，或将处理后的结果存回存储器。通过数据处理指令，我掌握了如何进行数据传送、算术运算（如加法、减法）和逻辑运算（如与、或、非）等操作。可以利用这些指令来操作寄存器中的数据，进行各种计算和逻辑判断。

2、 实验过程中出现的问题

首先，指令使用错误是常见的问题之一。由于汇编语言的复杂性，可能会在指令的书写或操作数的选择上出现错误。例如，错误的操作数或错误的寻址模式可能导致程序无法正常执行。解决这个问题的方法是仔细检查每个指令的书写和操作数的选择，确保它们与需求相匹

配。

其次，条件判断不准确也是一个常见问题。在实验要求中，根据条件执行不同的操作是很常见的情况。如果条件判断出现错误，可能会导致错误的代码路径执行，从而产生错误的结果。解决这个问题需要仔细检查条件判断语句，确保它们与预期的逻辑相符。

另外，循环控制问题也可能会出现。如果循环条件不正确或循环体内的更新操作有误，可能会导致循环无法终止或提前终止，从而导致程序无法按预期执行。解决这个问题需要仔细检查循环条件和循环体内的操作，确保它们满足预期的循环逻辑。